CODE SWEEP 审查报告（新模型）

> **范围：** 仓库 `tspike630/trade`，分支 `main`，提交 `2c6cd94`（含 PR #22–#26、#28）。 **日期：** 2026-10-13。 **方式：** 只读审查。先由 5 路子审查并行通读五层 + 看板/API；主审本人精读决策/执行/paper 关键路径，并把子审查的每一条结论回到 `file:line` 复核。4 个 P0 全部由主审在 `/tmp` 临时库上实跑复现。 **约定：**
>
> - 行号以 `2c6cd94` 为准。
> - "两树"指仓库根目录与 `crawler/` 下的同名副本。
> - 生产进程以 `crawler/` 为工作目录运行（`STABILIZE.md`、`DECISION_ENGINE_DRYRUN_24H.md`）。`crawler/execution` 是指向根 `execution/` 的软链。
> - 全文不含任何真实密钥值，假值写作 `<DUMMY>`，密钥位置写作 `<REDACTED>`。

一、总览

级别	数量	备注
P0（必改）	**4**	P0-1、P0-2 现网已生效。P0-3 在 paper 闸关闭时不触发。P0-4 只要用到 paper 定价（tick、CLI、Step B）就会触发。两者都必须在开 paper 闸前修
P1（应改）	**15**	并发与幂等 5 项、正确性 3 项、数据一致性 3 项、架构 2 项、方案对齐 2 项
P2（建议）	**38**	
观察项	**8**	需要现网数据才能定性或定量
合计	**65**	

按维度统计（P0+P1+P2，不含观察项）：

正确性	数据一致性	并发	安全/密钥	方案对齐	性能	架构	可维护性
20	10	7	5	5	3	2	5

一句话结论：

- **现网正在发生：**
  - Helius API key 会写进日志（P0-1）。

  - 指标层共用的 `updated_at` 时间戳让 JEV/X 覆盖过的 token 永久卡在 `tier='hot'`，决策层会对这些冻结数据持续发 buy（P0-2）。
- **开 paper 闸（Step B）前必须先解决：**
  - paper 价格源会冻结在第 500 行（P0-4）。
  - 复盘层把 paper 持仓的 UUID 当整数解析，会崩溃（P0-3）。
  - 执行链路既不原子也不幂等（P1-04、P1-05、P1-06）。

二、P0 清单

P0-1【安全/密钥】Helius API key 随 httpx 异常文本写入日志（现网生效）

- **位置：**
  - `crawler/sources/solana_helius.py:88`：`return f"https://mainnet.helius-rpc.com/?api-key={key}"`
  - `crawler/sources/solana_helius.py:236-237`：`resp = await client.post(url, json=payload)` 和 `resp.raise_for_status()`
  - `crawler/sources/solana_helius.py:485`：`tx = await rpc_get_transaction(sig, http_url=default_http_url(self.api_key))`
  - `crawler/sources/solana_helius.py:488-489`：`except Exception as exc:` 后执行 `log.warning("helius getTransaction failed sig=%s err=%s", sig[:16], exc)`
  - 输出目的地：`crawler/logging_setup.py:31-46`。logger `dexscreener.sources.helius` 同时写 stdout（systemd 收进 journald）和 `logs/crawler.log`（轮转保留）。
- **问题：** `httpx.HTTPStatusError` 的 `str()` 包含完整请求 URL，`?api-key=` 原样落盘。
- **触发条件：** 以下同时满足。
  - 运行现网 stabilize 命令 `main.py --helius-listen --sources x --loop`（`crawler/stabilize_run.sh:14`、`STABILIZE.md:3`）。
  - 已配置 `HELIUS_API_KEY`，未配置 `HELIUS_HTTP_URL`。
  - 某条通知的日志里提取不到 mint，于是改调 `getTransaction`，而该调用返回任意非 2xx（429 最常见）。
- **影响：** key 完整写入 `crawler.log*` 与 journald，任何能读日志的人或流程都能拿到。
- **复现（主审实跑）：** 用假 key 和 httpx `MockTransport` 返回 429，结果为 `key present in log line: True`。日志行如下：

  ```
  helius getTransaction failed sig=5555555555555555 err=Client error '429 Too Many Requests' for url 'https://mainnet.helius-rpc.com/?api-key=<DUMMY>'
  ```

- **修复建议（源码级）：**

```
# solana_helius.py _handle_notification
except httpx.HTTPStatusError as exc:
    log.warning("helius getTransaction failed sig=%s status=%s",
                sig[:16], exc.response.status_code)
except Exception as exc:  # noqa: BLE001
    log.warning("helius getTransaction failed sig=%s err=%s", sig[:16], type(ex
        c).__name__)

# logging_setup.py: 在 handler 级加脱敏(logger 级 filter 管不到子 logger)
_SECRET_RE = re.compile(r"(?i)(api[-_]?key=|/bot)[^&\s'\"/]+")
class _RedactFilter(logging.Filter):
    def filter(self, record):
        msg = record.getMessage(); red = _SECRET_RE.sub(r"\1<REDACTED>", msg)
        if red != msg: record.msg, record.args = red, ()
        return True
# console.addFilter(_RedactFilter()); file_handler.addFilter(_RedactFilter())
```

会话异常路径也要用同一正则脱敏：`:386-392` 记录 `exc`，`:416` 把 `err[:400]` 写进 `system_error` 告警（该类型总会推 Telegram）。

- **运维：** 无论查不查日志，都建议直接轮换 Helius key，并清理或脱敏历史 `crawler.log*` 与 journald（见 OBS-1）。

P0-2【数据一致性/正确性】指标层共用 `updated_at`：JEV/X 覆盖过的 token 离开 hot 后不再重算技术面，`tier` 永远停在 `'hot'`，决策层持续用冻结数据发 buy（现网生效）

- **位置：**
  - **到期判断读的是共用列：** `crawler/indicators/technical.py:386-399` 调 `get_indicator_updated_at`（`indicators/base.py:616-626`），若 `age < need` 就跳过。
  - **每次 upsert 都改写这一列：** `indicators/base.py:463-465`，`updated_at=excluded.updated_at`。
  - **funding 每轮覆盖：** `indicators/funding.py:150-155` 写 `"updated_at": utc_now_iso()`；`:202` 每轮遍历全部 `jev_signals`。
  - **sentiment 每轮覆盖：** `indicators/sentiment.py:374` 同样写 `updated_at`；`:429` 遍历全部 `x_watch`。
  - **只有 technical 写 `tier`：** `technical.py:283`。funding、sentiment、composite 只在 history 行里带 `tier`。
  - **决策层读取：** `crawler/decision_engine/core/data_adapter.py:144-149`，`WHERE tier = 'hot' ORDER BY COALESCE(jev_strength, -1) DESC`。
  - **JEV 只刷新采集层 hot token：** `sources/jev.py:239-261`、`:596`。
  - **线索表从不清理：** `jev_signals` 和 `x_watch` 在全仓没有任何 DELETE。

- **问题：** 指标循环每 300 s 一轮，warm 需 1800 s、cold 需 14400 s 才到期（`crawler/config.py:243-246`）。funding 和 sentiment 每轮都把共用的 `updated_at` 刷成当前时间，所以这类 token 的技术面永远不到期。结果是 `tier`、`change_*`、RSI/MACD/BOLL 全部冻结在最后一次 hot 时的值，`jev_strength` 也是旧值。
- **触发条件：** 默认配置下，任何曾被 JEV 拉过分或进过 `x_watch` 的 token，只要 `tokens.last_seen_at` 超过 1 小时（离开 hot）就会触发。受影响集合只增不减。
- **影响：**
  - `list_hot_tokens` 按 `jev_strength` 降序优先返回这批"僵尸 hot"行，挤占 `hot_limit=50` 的名额。
  - `jev_strength_v1` 等策略会对已冷却甚至已崩盘的 token 持续写 `decision_log` buy。
  - 下游 `jev_validity`、`factor_loop`、周报统计的都是这些信号；一旦开 paper，会直接按这些信号下单。
  - 如果以后把 `INDICATORS_WEIGHT_*` 调到 0 以上，`composite.py:183` 也会刷新全部行的 `updated_at`，所有 warm/cold token 都会被饿死。
- **复现（主审实跑）：** 两个 token 先处于 hot，之后都转为 warm 且价格下跌 46%；其中 tokena 有 `jev_signals`。

```
after passN: ('tokena','hot',  change_1h=15.58, rsi=100.0, jev_strength=0.48)
             ('tokenb','warm', change_1h=-39.1,  rsi=23.9)
decision_engine list_hot_tokens -> [('tokena', 'hot', 0.48)]   # 0.48 ≥ 0.3, jev_strength_v1 会出 buy
```

- **修复建议（源码级）：**
  1. **技术面独立时间戳：** `token_indicators` 新增 `tech_updated_at TEXT` 列，迁移写在 `indicators/base.py` 的 `_MIGRATE_TOKEN_INDICATORS`。`technical.compute_one` 写入 `out["tech_updated_at"] = now`，`run_batch` 的到期判断改读这一列。
  2. **其他族不再写共用列：** `upsert_token_indicators(row, touch_updated_at=False)`。funding、sentiment、composite 只写各自的 `jev_updated_at`、`sentiment_updated_at`、`composite_updated_at`。
  3. **决策层只取新鲜行：** `list_hot_tokens` 加条件 `AND tech_updated_at >= :now - 2*INDICATORS_HOT_INTERVAL_SEC`，或直接关联 `tokens.last_seen_at` 判断是否 hot，不再信任已冻结的 `tier`。
  4. **组合权重：** composite 开权重之前必须先完成第 2 步。
- **现网量化：** 用只读 SQL 统计僵尸行规模，见 OBS-2。

P0-3【正确性/崩溃】复盘层把 paper 持仓 UUID 当整数解析：开 paper 闸后，第一笔网关成交出现起，所有周报永久崩溃（开闸前必改）

- **位置：**

  - `crawler/review/data.py:241` 与 `review/data.py:238`：`"position_id": int(pr["id"]),`
  - 持仓查询 `crawler/review/data.py:218-224` 只过滤 `json_extract(meta_json,'$.decision_log_id') IS NOT NULL`，没有时间窗。
  - 只捕获了 `except FileNotFoundError`（`:252`）。
  - 写入方：`crawler/paper_trade.py:66` 定义 `id TEXT PRIMARY KEY`，`:135` 生成 `pid = str(uuid.uuid4())`。
  - `execution/executors/paper_executor.py:213-216` 把 `decision_log_id` 写进 `meta_json`。
- **问题：** UUID 字符串做 `int()` 抛 `ValueError`，被 `review/run.py` 外层捕获后以退出码 1 结束，不出报告。
- **触发条件：** 设置 `EXECUTION_PAPER_ENABLED=1`（Step B）后，PaperExecutor 开出第一笔带 `decision_log_id` 的仓位。查询没有时间过滤，所以之后任意 `--period` 都会失败。当前闸门关着，尚未触发。
- **影响：** 周一的复盘 routine（`WEEKLY_CHECK.md:47-51`）停摆，决策表与周报中断，只能手工删行恢复。冒烟夹具 `crawler/scripts/make_review_smoke_dbs.py:86` 用的是 `id INTEGER`，测不出这个问题。
- **复现（主审实跑）：** 用真实 `PaperLedger` schema 写入一笔网关风格的仓位，调用 `load_signal_chain`，结果为 `ValueError invalid literal for int() with base 10: '86f0f8a9-…'`。
- **修复建议：**
  - 改为 `"position_id": str(pr["id"])`，`SignalChainRow.position_id` 的类型改为 `Optional[str]`，并同步 `docs/REVIEW_R1_SPEC.md:234`。
  - 持仓查询加时间窗。入场成交改用 `WHERE position_id=? AND side='buy' ORDER BY id ASC LIMIT 1`；当前写法 `ORDER BY id DESC` 在平仓后取到的是卖出成交。
  - 冒烟夹具改用 `PaperLedger` 的真实 schema，并至少放一笔网关仓位。

P0-4【正确性】paper 价格源 `latest_price` 取的是"最早 500 行里的最后一行"：历史超过 500 行的 pair 价格永久冻结（开 paper 或运行 tick 前必改）

- **位置：**
  - `crawler/paper_trade.py:225-232`：`hist = store.list_pair_history(chain_id, pair_address, limit=500)`，随后 `for p in reversed(hist):`
  - `crawler/store.py:1272` 与 `execution/executors/paper_executor.py:85`：`sql += " ORDER BY recorded_at ASC LIMIT ?"`
  - `store.py:1276` 的 `purge_old_pair_history` 没有任何调用方；`HISTORY_RETENTION_DAYS`（`config.py:170`）也未被使用。
- **问题：** 查询返回的是该 pair 最早的 500 行，再取其中最后一行。一旦超过 500 行，返回值永远是第 500 行的价格。只有完全没有历史时才会回退到 `pairs.raw_json.priceUsd`。

- **触发条件：** 某个 pair 的 `pair_history` 超过 500 行。hot 节奏（300 s）约 41.7 小时达到，warm 约 10.4 天。现网 `pair_history` 共 1,131,648 行（`crawler/exports/reports/latest-hourly.json`）。
- **影响（所有 paper 定价都会错）：**
  - **退出判定：** `paper_trade.tick` 的 TP/SL/max_hold 判断与平仓价。调用方是 `paper_tick_loop.sh`（`RECOVERY.md:47`）和 `execution.run --paper-tick`。
  - **成交价：** Step B 下 `PaperExecutor` 的买卖价（`paper_executor.py:219`）。
  - **其他读价处：** `paper_trade.py status` 估值、`open` 默认价、`auto_hooks.py:156` 自动开仓。
  - **遗留回测：** `crawler/backtest.py:474` 和 `paper_trade.py:297` 的 simulate 只看每个 pair 最早 500 行。
  - **结果：** 价格冻结后 TP/SL 永不触发，每笔都在 max_hold 时以"冻结价"平仓，paper PnL 系统性失真，而这正是开实盘的裁决依据。
- **复现（主审实跑）：** 写入 600 行，价格依次为 1..600。`Store` 与 `_CrawlerRO` 两条路径都返回 `500.0`，应为 `600.0`。
- **性能附注：** 这不是全表扫描。`EXPLAIN` 显示 `SEARCH pair_history USING INDEX idx_pair_history_lookup`，但每次最多物化 500 行。
- **修复建议：** 在 `Store` 和 `_CrawlerRO` 上各加一个方法，`latest_price` 改为调用它。

```
def latest_pair_price(self, chain_id, pair_address):
    return self._conn.execute(
        "SELECT price_usd, recorded_at FROM pair_history "
        "WHERE chain_id=? AND pair_address=? AND price_usd IS NOT NULL "
        "ORDER BY recorded_at DESC LIMIT 1",
        (norm_chain(chain_id), norm_address(pair_address))).fetchone()
```

  - 同一个索引、单行读取，主审已验证返回 `600.0`。
  - 再与 `pairs.updated_at` / `priceUsd` 比较取较新的一方；超过 `max_age_sec` 时返回 None，按 `no_price` 拒绝成交（宁缺勿错）。
  - 把 `purge_old_pair_history(HISTORY_RETENTION_DAYS)` 接进主循环。

三、P1 清单

P1-01【架构】两树漂移，加上跨树混合导入

- **漂移现状（`diff -rq` 结果）：**
  - **decision_engine：** 5 个文件不同。根树 `core/data_adapter.py:38-39`、`backtest/replay.py:96-97`、`factors/buy/boll_breakout.py:18` 仍读 `boll_up` / `boll_low`，因为 PR

#28（`1e8e4bc`）只改了 `crawler/` 下的 3 个文件。
  - **review：** 6 个文件不同；`joint.py` 只在 crawler 有；`data.py` 根 577 行、crawler 1513 行。
  - **dashboard：** 3 个文件不同；`panels/evolution.py` 只在根有。
  - **scripts/jev_validity.py：** 两树的数据库与 CSV 路径解析不同。
  - **DECISION_ENGINE_PLAN03.md：** crawler 版缺少 Step 2/3 整节。
- **跨树混合导入：**
  - `execution/run.py:20-22` 用 `Path(__file__).resolve().parents[1]` 解析软链，把仓库根插到 `sys.path[0]`。因此在 `crawler/` 下运行 `python -m execution.run` 时，导入的其实是根树的 `decision_engine`。主审实测：`decision_engine imported: …/decision_engine/__init__.py`（根）。
  - 而 24h dry-run（`DRYRUN_24H.md:29-31`）导入的是 crawler 副本。
  - 根树 `decision_engine/config.py:37-41` 认 `DB_PATH` 环境变量，crawler 副本 `crawler/decision_engine/config.py:30` 只认 `DECISION_DB_PATH`。
- **文档互相冲突：**
  - `DECISION_ENGINE_RESTORE.md:5` 与 `RECOVERY.md:135` 说"以根为准"。
  - 生产 routine 却在 `crawler/` 下运行（`WEEKLY_CHECK.md:39-51`）。
  - `PLAN03:5` 写的"Both trees are updated together"已经不成立。
- **触发条件：** 按 RESTORE/RECOVERY 换机后走根树；或者按 `RECOVERY.md` §5 设置的 `DB_PATH` 指向 `crawler/dexscreener.db` 以外的文件。
- **影响：**
  - 根树的 boll 因子与回测里的 boll 列被 PRAGMA 过滤静默丢掉（`data_adapter.py:122`、`replay.py:110`），没有任何告警。
  - 根树复盘缺 joint、token_age、CCXT 基线；crawler 看板缺进化面板。
  - 设了 `DB_PATH` 时，网关读 `decision_log` 的库和决策引擎写 `decision_log` 的库可能不是同一个（split-brain）。
- **修复建议：**
  - 定一个唯一真源（建议用根树）。`crawler/` 下的同名目录改成软链（照 `crawler/execution` 的做法）或删除。
  - 收敛之前，先把 PR #28 移植到根树。
  - `_available_cols` 和 `_ind_cols` 发现请求的列不存在时打 WARNING。
  - 数据库路径统一由一个函数解析，`DB_PATH` 与 `DECISION_DB_PATH` 都认，并在启动时打印解析后的绝对路径。

P1-02【并发】没有任何单实例锁，`--once` 与 `--loop` 争同一个库：这就是 exit=124 的根因（任务单 §三-1）

- **位置：**

  - 全仓没有 `flock` / `fcntl` /pidfile。grep 只命中 `daily_check.py:822`，它只是展示 pid 文件。
  - `crawler/main.py` 没有 `--once`（参数定义在 `:120-160`，argparse 会直接以 2 退出）。
  - 带 `--once` 的入口是：`indicators/run.py:101`、`sources/jev.py:668`、`execution/run.py:51`、`decision_engine/run.py:79`、`sources/news.py:461`、`sources/ccxt/run.py:33`。
  - checkpoint 无属主、无租约：`store.py:650-663`；one-shot 运行可以接管 loop 正在进行的 cycle：`pipeline.py:647-651`、`:682-688`。
  - `main.py:188-193` 收到 SIGTERM 只设置 stop 标志；`pipeline.py:575` 在 `gather` 期间不检查这个标志（只在 `:536` 和 `:581` 检查）。
- **触发条件：** 任一入口的 `--once` 与它自己的 `--loop` 同时运行，例如 `timeout N python -m indicators.run --once`；或 `main.py` 的一次性运行与 loop 同跑。
- **影响：**
  - **exit 124 的含义：** 这是 GNU `timeout` 自己的超时退出码，不是程序的退出码。
  - **一次性运行为什么会超时：**
    - 打开 `Store()` 就要等写锁（见 P1-03）。
    - 每次提交都要争锁。IndicatorStore、JevStore、SignalBus 的 busy_timeout 只有 5 s。
    - 单次工作量没有上限（现网 pending 13,750 个）。
    - SIGTERM 要等 C 层等锁返回后才被处理。
    - 用 `timeout` 时没有加 `-k`，不会补发 SIGKILL。
  - **其他后果：** `execution.run --once` 与 `--loop` 会重复消费同一批 `decision_log`（见 P1-04）。`main.py` 的一次性运行会接管或提前结束 loop 的 checkpoint，并造成重复 enrich 与重复告警。
- **复现（主审实跑）：** 另一个连接持有 `BEGIN IMMEDIATE` 时，执行 `timeout 3 python -c "Store(db)"`，结果 `exit=124`。子审查另测了 `main.py --skip-lists` 的同一场景，结果为 exit=124，耗时 19.99 s，SIGTERM 之后日志仍打出 "Cycle completed successfully"。
- **修复建议：**
  - 每个常驻入口启动时先 `fcntl.flock(open(f"logs/{name}.lock","w"), LOCK_EX|LOCK_NB)`。`--once` 拿不到锁就以 75 退出，或改为给 loop 投递一个"立即跑一轮"的标志。
  - checkpoint 增加 `owner` 与 `lease_until`，更新时 `WHERE` 带上 owner。
  - `fetch_one` 内部检查 stop 标志，stop 时取消剩余任务；每轮设置批次上限。
  - 运维上统一用 `timeout -k 10 …`。

P1-03【并发/性能】每次打开 `Store()` 都执行两条无条件 `UPDATE`（会拿写锁），而 API、报告、巡检这些只读用途的进程都以读写方式打开主库

- **位置：**
  - `crawler/store.py:41-47` → `:134` → `:159-172`：每次打开都无条件执行 `UPDATE tokens …` 两次，没有像 `_migrate_lowercase_addresses`（`:194-199`）那样用 `schema_meta`

标记只跑一次。
  - **调用方：**
    - `api_server.py:59-60`，每个请求一次。
    - `api_server.py:334`、`:364` 用 `IndicatorStore`，会执行 DDL、`ALTER` 和 `commit`（`indicators/base.py:142-184`）。
    - `sources/solana_helius.py:497`，在 WebSocket 接收循环里每条通知一次；还有 `:403`。
    - `scripts/grok_report.py:169`、`tools/grok_context.py:166`、`scripts/grok_dd_report.py:398`。
    - `daily_check.py:1055`、`paper_trade.py:410`。
- **问题：** 在 SQLite 里，哪怕 `UPDATE` 一行都没匹配，也会开启写事务并持有 RESERVED 锁。
- **触发条件：** 任何其他写入者持有写事务时去打开 Store。
- **影响：**
  - 所谓"只读"API 在写负载下会阻塞最长 30 s，然后返回 500；同步的 `def` 路由还会占住线程池。
  - Helius 接收循环被阻塞，锁错误会直接拆掉 WS 会话，断线期间的通知全部丢失。
  - 与 P1-02 叠加。
  - `DB_PATH="dexscreener.db"` 是相对路径（`config.py:97`），在错误的工作目录下会新建一个空库并建全表。
- **复现（主审实跑）：** 零行 `UPDATE` 之后 `in_transaction=True`，第二个写入者得到 `database is locked`。在锁被占用时打开 `Store()` 会一直阻塞到 `timeout`。
- **修复建议：**
  - 回填迁移加一次性 `schema_meta` 标记。
  - 新增 `Store.open_ro(path)`：使用 `file:{abs}?mode=ro` 加 `PRAGMA query_only=ON`，不跑 `_init_schema`。API、Grok 报告、`daily_check`、paper CLI 都改用它。
  - Helius 的写库操作移出 WS 接收循环，改为队列加单一写线程。
  - 数据库文件不存在时直接报错（API 返回 503），不要隐式建库。

P1-04【并发/幂等】执行网关不原子、不幂等：成交、回执、游标、事件分四次提交；`decision_log_id` 没有唯一约束；`--since-id` 能把游标往回拨；循环没有异常保护

- **位置：**
  - `execution/gateway.py:479`：`exec_result = paper.execute(sig)`，写 paper 库并提交。
  - `:513-531`：`append_receipt`，单独提交。
  - `:532`：`set_last_processed_id`，单独提交。
  - `:533-545`：`_emit_order_events`，每个事件都新开一次连接（`order_events.py:62`）。
  - `execution/ledger.py:90-91`：`idx_exec_log_decision` 不是 UNIQUE。
  - `ledger.py:135-147`：游标直接覆盖写，不取 MAX。

  - `execution/run.py:60-65`：`--since-id` 的说明写着 "still advances ledger cursor"。
  - `:227-232`：`while` 循环没有 try/except。服务配置是 `Restart=always`（`deploy/dexscreener-execution.service:14-15`）。
- **触发条件：**
  - paper 成交之后、回执提交之前，进程崩溃、被杀或遇到锁错误。
  - 对生产 ledger 运行 README 里的冒烟命令 `--once --since-id 0`。
  - `RECOVERY.md:110-111` 的验证命令 `execution.run --once` 与常驻 loop 同时运行。
- **影响：**
  - 开 paper 后，同一条 decision 可能成交两次。
  - 回执重复。
  - 事件链被截断后永远不会补写，因为游标已经前进了。
  - 不开 paper 时，后果是回执重复和审计链断裂。
- **修复建议：**
  - 加 `CREATE UNIQUE INDEX ux_exec_decision ON execution_log(decision_log_id)`，处理之前先插一行 `status='claimed'` 抢占。
  - 回执、游标、事件放进同一个 ledger 事务；`record_event` 复用 `ledger.conn`。
  - 游标改为 `ON CONFLICT … SET value = MAX(CAST(value AS INTEGER), excluded.value)`；`--since-id` 只对本次运行生效，不持久化。
  - `PaperExecutor._buy` 先检查同一个 `meta.decision_log_id` 是否已有仓位，有就拒绝。
  - 循环包一层 try/except 加退避；单条坏消息记为 rejected 后继续前进。

P1-05【并发】PaperLedger 的现金与仓位更新不原子，且有多个进程同时写同一个 `paper_trading.db`

- **位置：**
  - `crawler/paper_trade.py:44`：默认 rollback journal，busy timeout 5 s。
  - `:107-112`：`set_cash` 自己 commit。
  - `:131-137`：先读现金，`set_cash` 先提交，然后才插入仓位。
  - `:173-180`：先查状态，`set_cash` 先提交，然后才 `UPDATE positions`。
  - `:183-185`：`UPDATE … WHERE id=?` 没有带 `AND status='open'`。
- **写入方：**
  - `execution.run`（PaperExecutor 加 `--paper-tick`）。
  - `paper_tick_loop.sh`（`RECOVERY.md:47`）。
  - crawler 里的 `auto_hooks`（`PAPER_AUTO_OPEN`）。
  - 命令行。
  - env 模板里 `PAPER_DB_PATH` 与 `EXECUTION_PAPER_DB` 都指向同一个 `paper_trading.db`（`crawler/deploy/dexscreener-crawler.env.example:67`、`:98`）。
- **触发条件：** 两个 tick 来源同时平同一个仓位；两次开仓并发；或在两次提交之间崩溃。

- **影响：**
  - 双重平仓会让现金重复入账；并发开仓会丢一次扣款。
  - 会出现"现金已扣但没有仓位"，或"现金已回但仓位仍为 open"（之后再平一次又会入账一次）。
  - 第二个平仓者抛 `ValueError('position not open')`，执行 loop 没有保护会直接崩溃，本次 tick 剩余的退出全部跳过。子审查已用确定性交错复现：两次平仓后现金为 11000，正确值应为 10000。
- **修复建议：**
  - 开仓、平仓各用一个 `BEGIN IMMEDIATE` 事务，`set_cash` 内部不再 commit。
  - 现金改为增量写法 `UPDATE account SET cash_usd = cash_usd + ?`。
  - 平仓用 `UPDATE positions SET … WHERE id=? AND status='open'`，确认 `rowcount==1` 后再记现金。
  - paper 库开启 WAL 并设置 busy_timeout。
  - 规定只能有一个 tick 来源（`paper_tick_loop` 与 `--paper-tick` 二选一）。

P1-06【正确性】卖出撮合按"同链同 token 最早的一笔"平仓，不分策略、不支持部分卖出；买入没有去重和冷却，开 paper 后同一个 token 每个 tick 都会加仓

- **位置：**
  - `execution/executors/paper_executor.py:312-338`：`_find_open` 只按 chain+token 或 chain+pair 匹配，不看 `strategy_id`。
  - `:370-374`：整仓平掉，忽略 `size_pct`。
  - `crawler/paper_trade.py:200`：按 `ORDER BY opened_at` 取最早一笔。
  - `decision_engine/core/engine.py:70-71`，加上 `core/risk_manager.py:13-17` 是直通。
  - `execution/position_manager.py:381-382`：`PM_ENABLED=0` 时全部放行。
  - 对比回测：`decision_engine/backtest/engine.py:174-178` 只匹配同一策略，`:247` 已持仓就跳过。
- **触发条件：** 开 paper 闸、PM 保持默认关。24h dry-run 每 60 s 会对同一批 hot token 重复发 buy（`DRYRUN_24H.md`）。
- **影响：**
  - 策略 B 的卖出会平掉策略 A、auto 或命令行开的仓位。
  - 部分卖出被当作全部平仓。
  - 同一个 token 每分钟开一个 5% 的仓位，直到现金耗尽。
  - paper 与回测的口径不一致。
- **修复建议：**
  - `_find_open` 增加 `json.loads(p["meta_json"]).get("strategy_id") == sig["strategy_id"]` 条件。
  - `size_pct<1` 时按比例平仓，或者直接拒绝。

- 决策层按 `(strategy, chain, token)` 加冷却（例如新增 `DECISION_COOLDOWN_SEC`），或者 Step B 默认打开 `PM_TOKEN_DEDUP`。

P1-07【正确性】回测重放有三类偏差：前视入场价、跌出 hot 的亏损仓按入场价平、同一轮多条残缺历史行

- **(a) 前视入场：**
  - `crawler/decision_engine/backtest/replay.py:333-339`：同一个 bar 里保留最后一行，却按 bar 起点时刻定价。
  - `:384-386`：`d = abs((pdt - target).total_seconds())` 在 ±1800 s 内双向取价，可以取到未来价格。
  - `:425`；`engine.py:207` 用 bar 起点附近的价格作为入场价。
- **(b) 亏损仓被抹平：**
  - `replay.py:408-431` 只给当前 bar 的 hot 前 50 行附价格。
  - 持仓的 token 不在列表里时，`engine.py:71-72` 取到 None，`execution/exits.py:65-67` 直接跳过，连 max_hold 都不触发。
  - 回测结束时 `engine.py:289-300` 按入场价以 `eod_flat` 平仓，PnL 记为 0。
- **(c) 残缺历史行：**
  - funding、sentiment、composite 每轮各写一条 history 行（`funding.py:169-187`、`sentiment.py:389-413`、`composite.py:201-223`），都不含 RSI/MACD/BOLL。
  - `base.py:526-532` 对缺失的键写 NULL。
  - `replay.py:339` 的 `bars[bar_ts][tok_key] = dict(r)` 让后写的行覆盖先写的行。
- **(d) 覆盖率前视：** 覆盖率取的是当前的 `jev_signals`（`replay.py:280-297`）。
- **触发条件：** (a) 任意动量类信号；(b) 持仓期间 token 跌出当期 hot 前 50，崩盘后很常见；(c) 任何有 `jev_signals` 或 `x_watch` 的 token。
- **影响：**
  - 回测收益系统性偏乐观。
  - 对 JEV 覆盖的 token，RSI/MACD/BOLL 因子在回测里全是 None，`factor_jev_rsi_and_v1`（AND 组合）永远不会触发。
  - `factor_loop --with-backtest` 的胜率与 PnL 也受影响。
- **复现（主审实跑）：** 一轮之后 tokena 有两条同一秒的 history 行：technical 行 `rsi=100`，funding 行 `rsi=None`。重放选中了后一条：`rsi_14 None macd None boll_upper None jev_strength 0.48`。
- **修复建议：**
  - bar 用结束时间戳。入场价取"不早于该指标行 `recorded_at` 的第一条快照"。
  - 持仓定价与 hot 列表解耦，按 pair 直接取 bar 时点及之前的最后一条快照；回测结束时按最后已知价格平仓。
  - 重放按 (bar, token) 对非空字段做合并（coalesce），或者让指标层每轮只写一条合并行。
  - 覆盖率按 as-of 时点取。

P1-08【正确性】技术指标"沿用旧值却刷新时间戳"，且变化率以最新快照为锚：陈旧数据被当作新鲜数据

- **位置：**
  - `crawler/indicators/technical.py:316-319`：本 tier 不计算的字段沿用 prev。
  - `:330-334`：pandas-ta 的 bar 不够时沿用 prev。
  - `:286`：整行 `updated_at=now`。
  - `:135`：`ref_now = now_epoch if now_epoch is not None else ep_now`，而 `:315` 调用时从不传 `now_epoch`。
  - `:274-276`：窗口内不足 2 个点时回退到全部历史。
  - `data_adapter.py:152-158`：没有 hot 行时跨 tier 回退取数。
  - `core/risk_manager.py:13`：直通，没有任何新鲜度检查。
- **触发条件：** token 从 hot 转 warm/cold（所有 token 都会，不限于 JEV 覆盖的）；或者 pair 快照停止更新。
- **影响：**
  - 冷 token 仍带着 hot 时期的 `change_5m`。
  - 用 3 小时前最后一次快照算出的 `change_5m` 被写上当前时间戳。
  - 违背了 `INDICATORS_PANDAS_TA.md:24` 的约定：历史不足时写 NULL，不伪造。
- **修复建议：**
  - 本轮没有计算的字段写 None，或者按字段族加 as-of 时间戳。
  - 调用 `change_pct(series, lb, now_epoch=time.time())`，快照过旧时直接返回 None。
  - 行内记录 `last_snapshot_at`，决策层按它（或 `tech_updated_at`）过滤。

P1-09【方案对齐】factor_loop 门槛与周检决策表不一致

- **位置：**
  - `review/factor_loop.py:66-73`、`:637-638`、`:657-670`。
  - `:82-83`：`factor_id TEXT NOT NULL UNIQUE`。
  - `:696-701`：`INSERT OR REPLACE`。
  - 对照：`DECISION_ENGINE_WEEKLY_CHECK.md:67-87`、`crawler/scripts/jev_validity.py:500`。
- **逐项对照：**

规则	决策表或周检代码	factor_loop	是否一致
n 门槛	`jev_validity` 用过滤后且有 1h 结果的 `st1.n ≥ 15`（`:500`、`:1095`）	`n_raw`，即过滤前的 n（`:638`，注释写着 "raw n 硬门槛"）	✗。现网 10-12 的数据：`n_raw` 为 40，过滤后为 14，门槛形同虚设

规则	决策表或周检代码	factor_loop	是否一致
弱势线	`<50%` 降级（`WEEKLY_CHECK:76`）	`<40%` 才判 fail，40–55% 判 `need_retest`（`:70`、`:664-670`）	✗。40–50% 本该降级，却记成复验
通过条件	连续两周 `≥55%`（`:75`）	单周 `≥55%` 就写 `status='accepted'`（`:657-662`）	✗（不会自动启用）
历史记录	需要逐周证据	`UNIQUE(factor_id)` 加 `INSERT OR REPLACE`，每周覆盖上一周，连人工写的 `activated` / `confirmed_at` 也被抹掉	✗
退役周数	N=2（`:84`）	`MAX_CONSECUTIVE_FAIL_WEEKS = 3`，而且只出现在 prompt 文本里（`:73`、`:341`）	✗

- **影响：** 交给人工把关的裁决标签是误导性的，"连续两周"也无从核验。系统不会自动启用策略（`auto_activate: False`），所以不是直接的交易风险。
- **修复建议：**
  - n 门槛直接复用 `jev_validity.decision_line` 的口径（`st1.n`）。
  - `GATE_UP_1H_RETRY` 改为 0.50，并新增"降级"分支。
  - 表结构改为 `UNIQUE(factor_id, week)`，写入用 `ON CONFLICT(factor_id, week) DO UPDATE`，只更新指标列，永远不覆盖 `status IN ('activated','retired')`。
  - 只有上一个 ISO 周同一 factor 也 pass，才写 `accepted`。
  - `MAX_CONSECUTIVE_FAIL_WEEKS` 改为 2。
  - 所有门槛常量收拢到一个模块，供 `jev_validity`、`factor_loop`、`evolution` 共用。

P1-10【方案对齐】jev_validity 的"连续 2 周"实际是"最近两次运行"（而且用全量样本），周门槛也不检查 n

- **位置：**
  - `crawler/scripts/jev_validity.py:686-696`：追加一行，并返回此前的所有行。
  - `:719-732`：`return week_gate_ok(prior[-1]) and week_gate_ok(current)`；`week_gate_ok` 只看上涨率与 `unique_token`。
  - `:121-131`：`load_buys` 没有时间窗。
  - `:1173-1177`。
- **证据：** 已提交的 `crawler/exports/reports/jev_validity_trend.csv` 里，10-10 与 10-12 两行完全相同（`40,14,9,0.714286,…`）。
- **触发条件：** `unique_token` 超过 10 且上涨率 ≥55% 之后，同一天重跑两次就满足"连续 2 周"。
- **影响：** Step 2 的硬门槛靠重跑就能满足；`n_1h` 不到 15 也能过。
- **修复建议：**

- 趋势 CSV 按 ISO 周 upsert，一周一行。
- 判定要求 `prior[-1]` 正好是上一个 ISO 周。
- `week_gate_ok` 增加 `n_1h >= MIN_RAW_N`。
- 如果坚持用全量统计，就要求两周之间确有新增样本。

P1-11【数据一致性】Helius/X 线索的 upsert 覆盖 DexScreener 的 token 画像（采集层的"`updated_at` 竞争"）

- **位置：**
  - `crawler/store.py:315-319`：`ON CONFLICT` 无条件覆盖 `raw_json`、`source`、`updated_at`、`last_seen_at`。
  - `sources/solana_helius.py:258-264`：写入占位 raw 和 `"source:helius"`。
  - `sources/x_api.py:323-330`；`pipeline.py:881` 会再写一次。
  - 消费方：`scoring.py:201-218` 调 `risk.py:133-137`，没有 links 且流动性低于 5000 时打上 `risk_no_socials` 并扣 5 分。
  - `alerts.py:77-85` 用质量分门控推送。
- **触发条件：** 已有 DexScreener 画像的 token 出现在任一 Helius 订阅程序的日志或 X 推文里。活跃 token 必然会遇到。
- **影响：**
  - links、description、icon 被占位数据覆盖，风险标签和质量分随之漂移，推送门控也跟着变。
  - `source` 溯源丢失。
  - `last_seen_at` 被线索不断刷新，tier 与 token 年龄的语义被污染。
- **修复建议：**
  - 线索来源的 upsert 只更新 `last_seen_at`；只有当原记录本身也是线索（`tokens.source LIKE 'source:%'`）时才覆盖 `raw_json`。
  - 或者把线索单独建表。
  - 量化方法：`SELECT source, COUNT(*) FROM tokens GROUP BY source`。

P1-12【并发/运维】Telegram 429 长时间封禁的状态只存在单个进程的内存里：跨进程或重启后仍会撞上限（任务单 §三-2）

- **位置：**
  - `crawler/alerts_telegram.py:305`：`ban_until: float = 0.0  # time.monotonic()`；另见 `:319-325`、`:390`。
  - `:820-824`：进程级单例。
  - `alerts.py:39-47`：`_TG_ALWAYS_PUSH_TYPES` 绕过所有推送过滤；`_tg_push_last` 只在内存里。

  - 直接调用发送的有 `scripts/grok_report.py:139-140` 和 `scripts/grok_dd_report.py:366`。它们绕过 `alerts.py` 的推送规则，但仍受本进程的发送闸门约束。
  - `arb_scan.py:186` 是独立进程。
- **触发条件：** 在 `TG_429_FIX.md` 记录的 44k–48k 秒（约 12–13 小时）长封禁期间，出现任何新进程第一次发送：systemd `Restart=always` 重启、cron 跑 Grok 报告、`arb_scan`、一次性运行等。
- **影响：** 每个新进程、每个 chat 至少再撞一次 429，可能延长封禁。单进程内部已经处理正确：长封禁直接丢弃，不 sleep、不重试（`:642-652`、`:683-698`）。
- **修复建议：**
  - `ban_until` 改存墙钟 epoch，放在共享位置：主库 `schema_meta` 的 `tg_ban_until:<chat>`，或带 flock 的 `logs/tg_ban.json`。
  - `_send_with_retries` 发送前先读这份共享状态。
  - Grok 报告改走统一的推送通道。

P1-13【数据一致性/数据丢失】`prior_ingest.py` 删除整个共享的 `prior.db`

- **位置：**
  - `crawler/scripts/prior_ingest.py:236-238`：注释写 `# 重建旁路库（不碰业务库）`，代码是 `if os.path.exists(DB):` 然后 `os.remove(DB)`。
  - `DB` 定义为 `data/prior/db/prior.db`（`:19`）。
- **问题：** 同一个库里还有其他脚本写的表：`ccxt_listing_watch.py:35`、`prior_tagger.py:50`、`prior_cex_listing_join.py:61`、`cex_scope_watch.py:92`。这与 `models/prior_tables.sql:64` 的注释"勿 DROP 整库以免擦掉其他 prior 表"直接冲突。
- **触发条件：** 重跑 prior 第二步的入库。
- **影响：** `cex_scope_snapshots`（每个 UTC 日一行）、`cex_listing_events`、`decision_prior_tags`、`baseline_history` 被清空，其中逐日快照事后无法重建。
- **修复建议：** 在一个事务内只 `DELETE FROM token_outcomes WHERE source IN ('solrpds','redpump')` 和对应的 provenance 行，再重新导入；不要删文件。

P1-14【数据一致性】文档里的 demo 命令默认往生产主库写假数据

- **位置：**
  - `crawler/backtest.py:582`（`--db` 默认是 `DB_PATH`）、`:599`、`:604-606`。
  - `crawler/paper_trade.py:331-335`、`:367`。
  - `crawler/demo_p5.py:88-89`（直接写 `ROOT/dexscreener.db`）、`demo_p3.py:63-64`、`demo_p4.py:98-99`。
  - 写入内容见 `backtest.py:537-569`。
  - README 里的命令：`crawler/README.md:185`、`:225-226`、`:237`、`:273-275`、`:416`（命令总索引）。

- **问题：** 这些命令会写入 `bttoken*` / `btpair*` 的 tokens、pairs 和 `tier='hot'` 的 `pair_history`，其中 `last_seen_at` 设为当前时间。全仓只有 `paper_trade.py:338` 认得 `btpair`，下游没有任何过滤。
- **触发条件：** 运维照 README 在生产目录执行 `python backtest.py … --seed-demo` 或 `python paper_trade.py demo`。
- **影响：** 假 token 依次进入指标层的 `token_indicators`、决策层的 hot 列表和复盘统计。
- **修复建议：**
  - demo 和 seed 命令强制要求显式 `--db`，或者默认写 `/tmp`。
  - 检测到目标是 `config.DB_PATH` 时拒绝执行。
  - 清理现网遗留：`DELETE … WHERE source='backtest_seed'`，三张表都要删。

P1-15【架构】看板的闸门指示灯显示的是看板进程自己的环境变量，不是网关的真实状态

- **位置：**
  - `dashboard/settings.py:118-121`：`paper = env_flag("EXECUTION_PAPER_ENABLED")` 和 `pm = env_flag("PM_ENABLED")`。
  - 渲染：`dashboard/app.py:1423-1425`。
  - 网关的实际状态来自命令行参数（`execution/run.py:93-125` 的 `--paper-enabled` / `--pm-enabled`）和 `crawler/.env`，而看板不加载 `.env`。
  - 实盘判定缺了 `LIVE_EXCHANGE_API_URL` 这一项（对照 `live_trade.py:220-221`）。
- **触发条件：** 用命令行参数开闸，或者网关与看板的环境变量不一致。
- **影响：** 安全指示灯可能显示 OFF，而网关实际是 ON。
- **修复建议：**
  - paper/PM 状态改为读最新一条 `execution_log.receipt_json` 里的 `paper_enabled` / `pm_enabled`（`gateway.py:437-438` 已经写入）。
  - 环境变量来源的值要标注"看板进程"。
  - `live_effective` 加入 URL 判定。

四、P2 与观察项

已知风险点核验（任务单 §三）

#	已知风险	结论	证据
1	`--once` 与 `--loop` 抢锁导致 exit=124	**未修**	全仓没有 flock 或 pid 锁。`main.py` 根本没有 `--once`。`Store()` 一打开就持写锁（`store.py:159-172`，实测零行 `UPDATE` 也持 RESERVED 锁）。`IndicatorStore`、`JevStore`、`SignalBus` 的 busy_timeout 只有 5 s。实跑：`timeout 3` 加上另一个进程持写锁，结果 exit=124。**需要加锁策略**，见 P1-02 和 P1-03
2	TG 429 长封禁治理：是否还有残留硬撞	**部分修复**	进程内正确：长封禁直接丢弃、不重试（`alerts_telegram.py:642-652`、`:683-698`）。残留：封禁状态只在单进程内存（`:305`、`:820-824`），重启或 cron 新进程每个 chat 至少再撞一次；always-push 类型绕过冷却（`alerts.py:39-44`）；Grok 脚本直接调发送。见 P1-12
3	`indicator_history` / `token_indicators` 短历史下 change/ic 的空值处理	**部分修复**	✓ `change_pct` 没有参照点时返回 None（`technical.py:121-142`）。✓ `spearman_ic` 在 n<3 或零方差时返回 None（`factor_layers.py:153-180`）；分层不足 10 个样本返回 None（`:116-117`）；`ic_summary` 不足 2 期返回 None。✓ NaN 和 inf 经 `safe_float` 置为 None（`crawler/review/data.py:95-104`）。✗ 历史不足时沿用旧值并刷新 `updated_at`（`technical.py:316-319`、`:330-334`）。✗ `change_*` 以最新快照为锚（`:135`）。✗ funding/sentiment 的 history 行技术列为 NULL。✗ JEV 缺分时 strength 记为 0.0 而不是 None（`funding.py:103-105`）。✗ `ic_summary` 只要 2 期就判定稳定，且用总体标准差。见 P0-2、P1-07、P1-08、P2-14、P2-25
4	`paper_trade.tick` 与回测 `_manage_exits` 是否已完全走 `execution/exits`（5-2b 之后两树是否一致）	**主路径已修，有残留**	✓ `paper_trade.tick` 调 `scan_exits`（`paper_trade.py:241-254`）。✓ 回测的 `engine._manage_exits` 已委托给 `scan_exits`（`engine.py:60-83`）；PR #26 两树一起改，两树 `engine.py` 一致，只是保留了这个函数名，内部不再有旧逻辑。✗ 遗留的 `crawler/backtest.py` 中 `backtest_tpsl` 仍有自己的退出实现（`:108-157`），被 `backtest.py --strategy tpsl` 和 `paper_trade.py simulate` 使用。✗ 语义仍有差别：回测的 `now` 用 `bar_ts`，无价格时跳过并以 `eod_flat` 按入场价平仓；paper tick 平仓不扣滑点，信号卖出却扣滑点；`time_exit` 卖出因子与 max_hold 功能重复。✗ 两树的 `replay.py` 不一致（boll 列名）

#	已知风险	结论	证据
5	看板 DataStore 以 `mode=ro` 直读 SQLite 是否全程只读	**已满足（仅限 DataStore 范围）**	业务库全部经 `connect_ro` 打开：`file:…?mode=ro`、`PRAGMA query_only=ON`、写语句正则拦截（`dashboard/data.py:58-61`、`:309-319`、`:365-368`），两树一致；`panels/risk.py:177` 也走 `connect_ro`。唯一读写的是聊天状态库 `dashboard_state.db`（`chat/sessions.py:34-35`、`:67-71`），但没有校验它不能指向业务库。注意：API（不是看板）以读写方式打开主库，并且每个请求都写（P1-03）

P2（建议，38 项）

安全与加固

- **P2-01** `crawler/api_server.py:49-55`：CORS 设置 `allow_origins=["*"]` 且 `allow_credentials=True`，会反射任意 Origin。`ALLOW_INSECURE_API` 两处解析不一致：`config.py:89-94` 接受 1/true/yes/on，`dashboard/settings.py:30-31` 只接受 `"1"`，注释却写着"Match api_server"。
  - 触发：不安全模式（未设 `API_SECRET` 且 `ALLOW_INSECURE_API` 为真）。
  - 影响：任意网站都能借运维人员的浏览器读取 API；`/health` 任何模式下都可跨域读取。
  - 修复：admin 页面是同源的，直接删掉 `CORSMiddleware`，或改成白名单并设 `credentials=False`；两处解析统一。
- **P2-02** `api_server.py:77`：`compare_digest` 收到非 ASCII 的 `X-API-Key` 会抛 TypeError，返回 500。修复：改为比较 bytes，即 `hmac.compare_digest(k.encode(), API_SECRET.encode())`；看板 `auth.py:64-69` 已经是这样处理的。
- **P2-03** `dashboard/app.py:880`：`SecretFilter` 挂在 root logger 上，而从子 logger 传播上来的记录不经过 logger 级 filter。子审查已实测。修复：挂到 handler 上。
- **P2-04** `crawler/live_trade.py`：
  - kill-switch 路径相对于当前工作目录（`:205-207`），换个目录运行就失效。
  - docstring 说该文件"present in repo"（`:7`），实际被 `.gitignore:13` 忽略。
  - `HttpStubAdapter` 用 `X-API-Secret` 头明文发送 secret（`:162-163`）。
  - 另外三道闸仍然有效，所以这是纵深防御层面的问题。修复：路径相对于模块目录；文件缺失时视为未授权；不要发送原始 secret。

并发与风控

- **P2-05** `PRAGMA busy_timeout = 5000` 覆盖了 `connect(timeout=30)`：`indicators/base.py:142-144`、`sources/jev.py:134-136`、`decision_engine/core/signal_bus.py:75-77`。锁超时后，JEV 结果被丢弃（`jev.py:564-567`），SignalBus 当轮剩余的信号全部丢失（`engine.py:100-105`）。修复：统一设为 30000；写失败时重试或暂存。
- **P2-06** 预交易风控对 SELL 也生效：`gateway.py:462` 加上 `risk/limits.py:46`，而 PM 对非 buy 一律放行（`position_manager.py:378-379`），所以持仓已达 max_open 的策略无法通过

信号平仓。
  - `daily_loss_limit` 从来没有传入（`gateway.py:292-296`），`limits.py:80-81` 因此恒为放行。
  - `EXECUTION_RISK_ENABLED` 和 `EXECUTION_EVENTS_ENABLED` 只认 `"1"`（`execution/config.py:12`、`:16`），与 `_env_bool`（`gateway.py:47-51`）不一致。
  - 风控默认关，开之前必须修。
- **P2-07** `position_manager.py:60-64`：`_pct_to_fraction` 在 1 处不连续。`PM_MAX_PER_TOKEN_PCT=1`（本意 1%）会变成 100%。PM 开启但没有账本时直接放行（`:384-390`）。修复：只接受百分点作单位，启动时打印解析结果；没有账本时拒绝（fail-closed）。
- **P2-21** 计数器有读改写丢更新：`store.py:488-495` 和 `:694-697`（cycle_number）。只入库不推送的告警在 `alerts.py:287-296` 提前返回，没有计数，所以计数 115,247 对不上 129,633 条告警。修复：改为单条 UPSERT 自增。

正确性

- **P2-09** `execution/exits.py:24`：`datetime.fromisoformat(str(s))` 在 Python 3.10 及以下不接受 `Z` 后缀，返回 None，max_hold 永远不触发；paper 的 `opened_at` 正是 Z 格式（`paper_trade.py:38`）。修复：先把 `Z` 换成 `+00:00`，与 `replay.py:26-27` 一致。现网版本见 OBS-5。
- **P2-10** paper 的估值与滑点口径不一致：
  - equity 按入场价估值（`paper_trade.py:215`），仓位大小和 PM 都忽略浮动盈亏。
  - tick 平仓用 mid 价、不扣滑点（`:251-253`），信号卖出却扣滑点（`paper_executor.py:369`）。
  - TP/SL 阈值基于含滑点的入场价，2% 滑点时 TP 需要 mid 上涨 12.2%。
  - 修复：统一滑点模型。
- **P2-13** `factors/buy/rsi_bounce.py:15`、`:32`：默认 `min_change_5m=0.0` 且用 `<` 判断，完全横盘（c5=0.0）也被当作"反弹确认"。MACD 线与 signal 线可能来自不同轮次（`technical.py:328-334` 逐键沿用旧值，`macd_cross.py:28` 直接比较）。修复：改用 `<=` 或正阈值；MACD 三个值整组更新或整组置空。
- **P2-15** 情绪面问题（权重为 0 的观察通道）：
  - spike 实际反映的是 watch 的年龄和搜索节奏（`sentiment.py:104-112`、`:150-156`）。
  - 增长基线选到了同一秒的技术行，恒为 None（`indicators/base.py:350-357`，没有非空过滤）。
  - 极性把 token 自己的 symbol 也当文本计分（`base.py:315-318`）。
  - X 搜索单页 `max_results=10`，倍数阈值 3（`config.py:74`、`:78-79`）：前一窗口 ≥4 次提及的 token 永远不会 spike（`x_api.py:216`、`:396-408`、`:135-146`）。
  - "价格+社交"双信号升级因 pair_address 匹配不上，基本不会触发（`x_api.py:434-442`）。
- **P2-16** `indicators/base.py:365-380`：主 pair 按历史最大流动性选，而 docstring 写的是最新流动性。结果技术面和回测都用已经枯竭的池子定价。修复：按各 pair 最新快照的流动性排序。
- **P2-24** factor_loop：

  - `--dry-run` 仍会新建 `factor_candidates.db`，并覆盖正式周报 md（`factor_loop.py:748`、`:184-186`、`:786`）。
  - `_latest_ic` 取的是 CSV 最后一行（`:284-297`）。按 `factor_ic_history.py:76` 排序后，最后一行是 `volume_strength`，既不是 JEV 也不是均值，而且根本没有 `ir` 列。
  - `_factor_id` 舍入后会碰撞，导致 LLM 提出的候选被丢弃（`:201`、`:386-389`、`:501`）。
- **P2-25** IC 稳定性被高估：
  - 每天跑 7d 滚动窗口，并以日期为键（`review/run.py:370`），相邻窗口约重叠 6 天。
  - `ic_summary` 只要 2 期就判定，并用总体标准差（`factor_ic_history.py:94-99`、`:108`）。
  - IC 把买卖方向和所有策略混在一起算（`factor_layers.py:109-115`）。
  - 修复：每个 ISO 周一条，用样本标准差，至少 4 期，按策略和买入方向分别计算。
- **P2-27** `review/run.py:70-81` 的 `--start` / `--end` 归一化：带 `+08:00` 的输入原样返回，再与 Z 格式字符串做字典序比较（`data.py:153`、`:156`），窗口偏 8 小时；空格分隔的时间会被拼成非法字符串。修复：`fromisoformat` 解析后转 UTC，再用 `to_iso_z` 输出。
- **P2-29** Grok 增量模式：
  - 低分 token 恒被丢弃：低分查询缺 `scored_at` 列（`tools/grok_context.py:47-52`），`scripts/grok_report.py:71-80` 也没有回退。
  - 在错误的工作目录下会新建空主库，根树 `scripts/jev_validity.py:34-36` 会把它当主库读。
- **P2-31** 看板聊天：
  - 历史取的是最早的 200 条（`chat/sessions.py:137-143`，`ORDER BY id ASC LIMIT`），超过 200 条后新问题进不了上下文（`page.py:450-456`）。
  - 工具只看最近 100 行，chain 大小写敏感，异常被吞成"暂无数据"（`chat/tools.py:232-241`、`:308-311`）。
- **P2-35** JEV：
  - `Retry-After` 没有上限，直接 sleep（`jev.py:464-477`）。
  - pending 文件读完就截断，与追加方竞争；超出 limit 的条目在截断后丢失（`jev.py:514`、`:538`、`:609`；`auto_hooks.py:64-67`；只在 `JEV_ON_NEW_LISTING=1` 时出现）。
- **P2-36** `store.py:945-952`：X watch 列表按先进先出，永不过期。小时预算 20 次，新 token 永远排不上搜索。
- **P2-37** `sources/solana_helius.py:58`、`:138-163`：用正则从日志文本里提取"mint"，产生大量假地址，浪费 enrich 并制造垃圾告警。快照里最新的 26 条 `new_listing` 全部来自 Helius 线索。

数据一致性

- **P2-11** 回测与实时运行的参数和成本口径不同：
  - 回测阈值默认 0.15/2.0/0.5（`backtest/run.py:166-168`），实时为 0.3（`crawler/decision_engine/config.py:40`），combined 策略为 0.15/1.0/0.3（`combined_strategy.py:21-23`）。

  - 回测有 `min_liq=50k` 过滤（`filters.py:26`），实时没有。
  - 回测不计手续费和滑点（`ledger.py:348-349`）。
  - 收尾 PnL 两套算法：metrics 用标记市值（`metrics.py:59`），attribution 汇总已实现盈亏（`attribution.py:470`），同一次运行会分别报出 +100 美元和 0 美元。
- **P2-14** JEV 错误行覆盖上一份好数据：`jev.py:185-190` 用 `ON CONFLICT` 覆盖全部列，错误行见 `:505`；随后 `funding.py:103-105`、`:110` 把它变成 neutral、strength 0.0，并刷新时间。修复：出错时只更新 `error` 和 `fetched_at`，数据列用 COALESCE 保留；没有分数时 strength 设为 None。
- **P2-17** `sources/ccxt/core.py:158-172`：正在形成的 K 线用 `INSERT OR IGNORE` 永久写死（`models/ccxt_tables.sql:3` 的注释却称"幂等 upsert"）；一个交易所的写事务横跨所有 `fetch_ohlcv` 网络调用。修复：改为 `ON CONFLICT(exchange,symbol,timeframe,ts) DO UPDATE`，或丢弃未收盘的 bar；每个 symbol 提交一次。
- **P2-26** 前向收益的对齐与口径：
  - p0 在 ±30 分钟内双向匹配，可能取到信号之后的价格（`review/data.py:318-323`、`jev_validity.py:229-234`）。
  - 覆盖率和流动性取当前快照，属于前视。
  - R1 所谓"同口径"实际与 jev_validity 不同：`data.py:332-336` 读的 `raw.liquidity` 从来没被写入过，覆盖率查询不带 `chain_id`（`:436-441`）。
  - `unique_token` 也统计了没有 1h 结果的 token（`jev_validity.py:603`）。
  - 目前数值相同（n=14，unique 9，71.4%），历史超过 7 天后会分叉。
- **P2-30** 看板拒绝分布只看 `validation_ok` / `validation_errors`，不看 `status`，所以 PM、风控、paper 失败的拒绝不被统计（`dashboard/data.py:1071-1083`、`:650`；`panels/execution.py:13`、`:27`）。另外 `validation_errors` 存的是 JSON 数组（`execution/ledger.py:176-180`），读取方却按 `;` 切分（`dashboard/parsing.py:152`、`crawler/review/data.py:869`）。
- **P2-34** 文件位置与地址格式：
  - `news_events.db` 的位置随代码树变化：`sources/news.py:40-41` 两树代码相同，`RECOVERY.md:91` 从根运行，所以写到根目录；`crawler/review/evolution.py:57` 却读 crawler 目录。
  - Solana base58 地址被统一转小写作为主键（`store.py:20-22`）。JEV 调用前会从 `raw_json` 恢复大小写（`jev.py:320-360`），所以不影响 JEV 请求，但主键本身不是合法的链上地址。

方案对齐

- **P2-08 order_events 状态链（任务单 §二.7）：** 网关内各路径的状态链是写全的：`new` → `validated` → `submitted` → `filled`，各阶段的 `rejected` 带 stage 说明（`gateway.py:298-399`）。但还有以下缺口：
  - 事件与 `execution_log` 不在同一事务，且在游标前进之后才写（见 P1-04）。

  - `cancelled` 从未产生；docstring 写的 `accepted` 不在 `EVENTS` 里（`order_events.py:3`、`:23`）。
  - TP/SL/max_hold 由 paper tick 平仓，不经网关，所以平仓这一腿既没有 `execution_log` 行，也没有任何事件（`paper_trade.py:241-254`）。
  - 复盘和看板都不读 `order_events` 和 `status`。
  - 修复：平仓腿也走网关，生成回执和事件；事件与回执放在同一事务。
- **P2-12** 因子卖出契约：`FactorStrategy` 发出的 sell 不带 `pair_address`（`factor_strategy.py:87-92`），而网关要求必填（`gateway.py:33-40`、`:124-127`）；一旦接入实时持仓，这些 sell 全部校验失败。
  - `trail_stop` 依赖 `data["prices"]` 和 `peak_price`，实时和回测都不提供，是死因子（`trail_stop.py:19-23`；`PLAN03:59` 已承认）。
  - `time_exit`（单位秒）与 max_hold（单位分钟）功能重复。
- **P2-38** 网关在 Step B 不区分 dry_run 采样信号：周一三五 0.15 阈值的采样与 24h 常驻的 0.3 阈值 dry-run 使用同一个 `strategy_id` 和同一个 `mode`（`WEEKLY_CHECK.md:22`、`:59`），开 paper 后采样信号也会成交。另外，新建或迁移的 ledger 游标从 0 开始，会重放全部历史 `decision_log`（`gateway.py:259` 没有时效过滤；`ledger.py:128-129`）。修复：信号增加 purpose 字段；ledger 首次创建时把游标设为 `MAX(id)`；加最大信号年龄。

性能

- **P2-18** 没有数据保留策略：`pair_history`（113 万行，purge 无人调用）、`indicator_history`、`alerts`（12.9 万）、`decision_log`、`x_watch`、`run_checkpoint` 从不清理。索引缺口：
  - 告警去重对列套了 COALESCE，索引只能用到 `type`（`store.py:470-480`）。
  - `/status` 做全表 COUNT（`store.py:1398-1401`）。
  - `indicator_history` 只有 `(chain_id, token_address, recorded_at)` 索引（`indicator_tables.sql:96-99`），重放、看板、复盘按时间或 tier 查询都是 SCAN。
  - `execution_log` 没有 `ts` 索引（`ledger.py:90-93`）。
- **P2-19** 看板每个客户端每 30–60 秒在事件循环上同步做一次全量刷新，包括隐藏的页签（`dashboard/app.py:1169-1182`、`:1276`），其中有多处全表扫描（例如 `data.py:1319-1320`）。`/dashboard/summary` 每次对所有 pairs 做两遍 `json.loads`（`store.py:1338-1349`、`api_server.py:268-269`）。修复：共享快照缓存，用 `run.io_bound`，只刷新可见页签。
- **P2-20** 采集队列：`_build_queues` 的去重是 $O(n^2)$（`pipeline.py:487-493`）；pending 全表载入，没有批次上限（`pipeline.py:506-511`、`store.py:538-542`）。

可维护性与运维

- **P2-22** 大约 15 个入口共用同一个 `RotatingFileHandler` 文件（`logging_setup.py:38-43`），各进程各自轮转，日志会丢失或写错文件。修复：systemd 下只写 stdout，或用 `WatchedFileHandler` 加 logrotate。
- **P2-23** 备份只覆盖主库（`backup.py:39`、`:52-57`），而 `RECOVERY.md:81` 列的是四个库。

- **P2-28** 复盘层：R3 优化闸门从命令行永远打不开（`review/run.py:289` 把 `weekly_check_passed=False` 写死；`optimizer.py:421` 用窗口内跨度当作历史长度）。报告里有硬编码的过期事实：`report.py:960` 的"paper 仍为 0"、`:271` 和 `:999` 的"~0.51d"、`:872` 的"71.4%/57.1%, n=14"、`evolution.py` 中多处。
- **P2-32** 看板杂项：
  - 回测权益的颜色按写死的 10000 判断（`dashboard/app.py:1981`）。
  - 看板解析 DB 路径用的环境变量与写入方不一致（`dashboard/data.py:96-97` 对照 `execution/ledger.py:32-44`、`execution/run.py:36-37`）。
  - 心跳过期阈值：API 写死 120 s（`crawler/config.py:49`），看板为 900 s。
  - Top gainers 在 h24=0 时回退到 h6，却仍标为 h24（`store.py:1356`、`:1387`）。
- **P2-33** 死代码和残留实现：
  - `technical_lib.compute_technical_last`（`:184-189`）没有调用方，仍输出 `boll_up` / `boll_low` 别名，可能是历史命名混乱的源头。
  - `alerts_telegram._notify_admin_send_failure`（`:781`）没有调用方。
  - 代理健康检查线程每轮新建一个（`pipeline.py:711`），而 `proxy_pool.stop_health_loop` 没有调用方（`proxy_pool.py:185-200`）；只在 `USE_PROXY=1` 时出现。
  - `proxy_pool_low` 告警直接写 `delivered=True`，实际从未发送（`proxy_pool.py:283-299`）。
  - 看板注册了却从不渲染的面板（`panels/registry.py:29-34`）。
  - 遗留的 `crawler/backtest.py` 中 `backtest_tpsl` 仍有自己的退出逻辑（`:108-157`）。
  - 回测里的 `_manage_exits` 只保留了名字，内部已委托给 `scan_exits`，不属于旧逻辑残留。

观察项（8 项；需要现网数据，全部给出只读验证方式）

- **OBS-1** 现网日志里是否已经有 Helius key。只做计数、不打印内容：`grep -c 'api-key=' logs/crawler.log*`。结果大于 0 就必须轮换；建议无论结果如何都轮换。
- **OBS-2** P0-2 僵尸行的规模。只读 SQL：

```
SELECT COUNT(*) FROM token_indicators ti
JOIN tokens t ON t.chain_id=ti.chain_id AND t.token_address=ti.token_address
WHERE ti.tier='hot' AND t.last_seen_at < strftime('%Y-%m-%dT%H:%M:%SZ','now','-1
        hour');
```

  再统计近 7 天有多少 `decision_log` buy 命中这些 token。
- **OBS-3** `paper_trading.db` 现状：是否有 open 仓位、`paper_tick_loop` 是否在运行、open 仓位对应的 pair 中有多少历史超过 500 行。用来判断 P0-4 是否已经产生了错误的 PnL。
- **OBS-4** 锁等待的频率：日志中 `database is locked`、`jev upsert failed`、`indicator_history write failed`、`tick crashed` 各出现多少次。用来决定 busy_timeout 的统一值和批量事务的优先级。

- **OBS-5** 生产环境的 Python 版本。低于 3.11 时 P2-09 升为 P1，因为 paper 的 max_hold 会完全失效。
- **OBS-6** Telegram 跨进程撞上限的实际次数：按进程统计 `429_long_ban_no_retry` 和 `telegram 429` 日志，并看封禁是否因此被延长。用来量化 P1-12。
- **OBS-7** 现网 `ALLOW_INSECURE_API` 与 `API_HOST` 的取值，决定 P2-01 的 CORS 风险是否真的可被利用。
- **OBS-8** `indicator_history` 与 `pair_history` 的日增量、看板 `slow read` 告警的频率、看板运行用户是否对 `crawler/` 有写权限（`mode=ro` 读 WAL 库需要创建 `-shm` 文件）。用来决定 P2-18、P2-19 的优先级。

五、Top 5 升级建议（架构级）

> 工作量按改动面、侵入度和依赖风险描述，不估日历时间。

1. **消除两树，确立单一真源，并加 CI 守卫。**
   - **理由：** 对应 P1-01。PR #28 只修了一树；网关与决策引擎导入的不是同一份代码；文档各说各的。
   - **做法：**
     - 以根树为真源，`crawler/` 下改为软链。
     - 先补齐两树的差异：根缺 boll 修复，crawler 缺 `DB_PATH` 支持，根的 review 缺 joint/token_age/CCXT，crawler 的看板缺进化面板。
     - CI 增加 `diff -rq` 检查和导入路径断言。
   - **工作量：** 6 组目录（decision_engine、review、dashboard、`scripts/jev_validity.py`、`sources/news.py`、docs），中等侵入。
   - **风险：** 生产以 `crawler/` 为工作目录，依赖不少相对路径（`news_events.db`、`factor_candidates.db`、trend CSV、`exports/`），数据文件要一起迁移。
2. **主库并发治理：单实例锁、只读工厂、统一连接策略。**
   - **理由：** 对应任务单 §三-1，以及 P1-02、P1-03、P2-05。大约 10 个进程写同一个 SQLite 文件。
   - **做法：**
     - 所有常驻入口加 `flock`。
     - 新增 `Store.open_ro` 只读工厂；迁移改为一次性执行。
     - 写入方统一 `busy_timeout=30000`，逐行提交改为分批事务。

    - 评估把 `decision_log` 迁到独立的 `decision.db`，主库只留采集层与指标层两个写入方。
  - **工作量：** 连接层（`store.py`、`indicators/base.py`、`sources/jev.py`、`signal_bus.py`）、约 10 个入口加锁、API/报告/巡检改为只读，中等侵入。
  - **风险：** 迁移标记与 WAL checkpoint 需要回归测试；`decision_log` 迁库会影响网关、复盘、看板的读取路径。

3. **执行链路事务化与幂等化（开 paper 闸的前置条件）。**
  - **理由：** 对应 P0-4、P1-04、P1-05、P1-06、P2-08。
  - **做法：**
    - 按 `decision_log_id` 唯一抢占；paper 侧也按 `decision_log_id` 唯一。
    - 回执、游标、事件放进同一个 ledger 事务。
    - PaperLedger 改为原子写：`BEGIN IMMEDIATE`、增量现金、条件更新。
    - 卖出按策略撮合；平仓腿也走网关，生成回执和事件；只保留一个 tick 来源；价格源改为 `DESC LIMIT 1` 加最大年龄。
  - **工作量：** `execution/gateway.py`、`ledger.py`、`order_events.py`、`paper_executor.py`、`crawler/paper_trade.py`，中高侵入，涉及 schema 迁移（唯一索引、事件表）。
  - **风险：** 需要按 `execution/README.md` 把 Step A/B/C 冒烟全部重跑一遍，验证行为等价。

4. **指标层改为"按族时间戳 + as-of 语义"，并修正回测重放。**
  - **理由：** 对应 P0-2、P1-07、P1-08、P2-13、P2-16。
  - **做法：**
    - 技术面、资金面、情绪面、组合各用自己的 `*_updated_at`。
    - history 每轮只写一条合并行，或在重放时合并。
    - `change_*` 以当前时间为锚；决策层只消费新鲜行。
    - 重放用 bar 结束戳；成交价取信号时点之后的价格；持仓定价与 hot 列表解耦；收尾按最后已知价平仓。
  - **工作量：** `indicators/*`（technical、funding、sentiment、composite、base）、`indicator_tables.sql` 迁移、`data_adapter`、`replay` / `engine`，中等侵入。
  - **风险：** 历史 `indicator_history` 已是多行格式，读取端要兼容新旧两种；回测结论会整体变化，需要重新建立基线。

5. **门槛与口径收拢到一个模块，数据按周建模。**
  - **理由：** 对应 P1-09、P1-10、P2-24 至 P2-28。jev_validity、factor_loop、evolution、review 四处各写各的门槛，"连续两周"和 IC 周期都不按周记录。
  - **做法：**
    - 新增 `review/gates.py`，集中 n 口径、55%/50% 线、连续 N 周判定。
    - 趋势 CSV 与 `factor_candidates` 改为以 (factor, ISO 周) 为主键。

    - IC 历史按 ISO 周记录、互不重叠。
    - 报告里的数字全部从计算值渲染，去掉硬编码。
  - **工作量：** `review/factor_loop.py`、`scripts/jev_validity.py`、`review/evolution.py`、`review/report.py`、`review/factor_ic_history.py`，侵入度低到中。
  - **风险：** `INSERT OR REPLACE` 已经丢掉的历史周记录无法恢复，只能从各周 md 报告手工回填。

六、审查范围与未覆盖项

- **主审亲自精读：**
  - `execution/` 全部文件（gateway、run、ledger、exits、order_events、position_manager、config、risk/*、paper_executor）。
  - `crawler/paper_trade.py`。
  - `crawler/decision_engine/` 的 core/*、`strategies/__init__.py`、`factor_strategy.py`、`run.py`、`config.py`、`backtest/engine.py`、`backtest/replay.py` 主体；根树的差异文件。
  - 指标层：`indicators/run.py`、`technical.py` 主体、`funding.py`、`base.py` 相关段。
  - `crawler/store.py` 相关段、`history.py`、`alerts.py` 推送策略段、`alerts_telegram.py` 429 段、`solana_helius.py` 相关段。
  - 复盘层：`review/factor_loop.py` 的门槛与入库段、`factor_layers.py`、`factor_ic_history.py`、`crawler/review/data.py` 链路段。
  - 看板与 API：`dashboard/data.py` 连接段、`dashboard/settings.py` 闸门段、`crawler/api_server.py` 鉴权与 CORS 段。
  - 文档与部署：`RECOVERY`、`STABILIZE`、`WEEKLY_CHECK`、`PLAN03`、`RESTORE`、`DRYRUN_24H` 等文档和 deploy 单元文件。
- **子审查精读、主审复核：** 5 路并行，主审逐条回到 `file:line` 复核；文中所有行号都已在 `2c6cd94` 上重新确认。
  - 采集层全量：main、store、config、client、pipeline、sources/、*ccxt/*、heartbeat、backup、history、alerts*、proxy_pool、logging、worker、models、deploy。
  - 指标层全量：indicators/*、scoring、risk。
  - 复盘层：两树的 review/*、两树的 jev_validity、grok 脚本。
  - 看板与 API 全量：含两树 diff、`static/admin.html`。
  - 决策、执行、paper、回测全量：含 `live_trade.py`、遗留 `backtest.py`、`auto_hooks.py`。
- **未覆盖或仅抽查：**
  - 离线研究脚本 `crawler/scripts/prior_*.py`（只核对了 `prior_ingest` 删库那一段）、`self_survival.py`、`cex_scope_*`、`ccxt_listing_watch.py`（只确认它写 `prior.d`

b）。
  - `crawler/review/evolution.py` 约第 420–560 行和第 600–1660 行。
  - `crawler/daily_check.py` 的大部分。
  - `crawler/demo_*.py`（只核对写库的默认值）。
  - `crawler/worker/`（未部署）。
  - `crawler/arb_scan.py`。
  - 各阶段 `*_RESULT.md` 结果文档未逐篇核对。
  - 未合并的远端分支（例如 `cursor/indicators-atr-adx-v2-e807` 里的 ATR/ADX）不在范围内；`main` 上没有计算 ATR/ADX（`technical_lib.py:102`）。
- **没做、也做不了的：**
  - 本机没有现网四库（`*.db` 被 `.gitignore` 忽略）。所有规模和频率相关的判断都标为观察项，并给出只读 SQL 或 grep 验证方法。
  - 现网规模数字（113 万行 `pair_history`、12.9 万条告警、13,750 个 pending 等）来自已提交的 `crawler/exports/reports/latest-hourly.json`（2026-10-13T01:47Z）。
  - 没有启动常驻服务做压测。
  - 复现环境：Python 3.12.3、SQLite 3.45.1；全部基于 `git archive` 导出到 `/tmp` 的副本和临时库。

附：验收自检（V1–V5）

#	检查项	结果
V1	报告完整	✓ 第一至第六节齐全。总数 65（P0 4 / P1 15 / P2 38 / 观察项 8），附维度统计
V2	每条可追溯	✓ P0/P1 每条都有位置（`file:line`）、触发条件、影响、源码级修复；P0 另附实跑复现。P2 和观察项为压缩格式，但都含 `file:line` 和修复方向
V3	已知风险点逐条核验	✓ §三的 5 条都给出了"未修/部分/已修"结论和证据，见第四节开头的表格
V4	没有改动任何代码	✓ 审查全程 `git status --porcelain` 为空，`git status --ignored` 也没有新增的 `__pycache__` 或 `*.db`。所有 Python 都带 `PYTHONDONTWRITEBYTECODE=1`，运行在 `/tmp` 副本上。本机不存在业务四库，"业务库 mtime 不变"自然成立；mtime 有变化的只有编辑器自身的 `state.db`，不是业务库
V5	不打印密钥	✓ 报告中出现的 key/token/secret 只是字段名或概念名，没有任何真实值。复现用的是假值，以 `<DUMMY>` / `<REDACTED>` 占位